

Resampling1D

Resampling 1D

```
# -*- coding: utf-8 -*-
"""
Created on Mon Jan 19 17:51:11 2026

@author: rslim
"""

import cv2
import numpy as np

class Upsample1d:
    def __init__(self, upsampling_factor):
        self.upsampling_factor = upsampling_factor

    def forward(self, x):
        N, C, input_width = x.shape
        output_width = self.upsampling_factor * (input_width - 1) + 1
        z = np.zeros(shape=(N, C, output_width), dtype=np.float64)
        for i in range(0, output_width, self.upsampling_factor):
            z[:, :, i] = x[:, :, i // self.upsampling_factor]
        return z

    def backward(self, dLdz):
        N, C, output_width = dLdz.shape
        input_width = (output_width - 1) // self.upsampling_factor + 1
        dLdx = np.zeros(shape=(N, C, input_width), dtype=np.float64)
        for i in range(0, output_width, self.upsampling_factor):
            dLdx[:, :, i // self.upsampling_factor] = dLdz[:, :, i]
        return dLdx

class Downsample1d:
    def __init__(self, downsampling_factor):
        self.downsampling_factor = downsampling_factor

    def forward(self, x):
        N, C, input_width = x.shape
        self.original_input_width = input_width
        output_width = (input_width - 1) // self.downsampling_factor + 1
        z = np.zeros(shape=(N, C, output_width), dtype=np.float64)
        for i in range(output_width):
            z[:, :, i] = x[:, :, i * self.downsampling_factor]
        return z

    def backward(self, dLdz):
        N, C, output_width = dLdz.shape
        dLdx = np.zeros(shape=(N, C, self.original_input_width), dtype=np.float64)
        for i in range(output_width):
            dLdx[:, :, i * self.downsampling_factor] = dLdz[:, :, i]
        return dLdx

def test_upsample1D(N, C, W, S):
    upsample = Upsample1d(S)
    # Insert S-1 columns between each original columns.
```

```

# Therefore, the output will be of size  $W + (S-1)*(W-1) = S*(W-1) + 1$ 
#  $W = 5$ ,  $S = 3$ , then  $\text{output\_size} = 5 + 2 * 4 = 13$ .

input_tensor = np.random.uniform(0, 5, (N, C, W))
print("input_tensor.shape", input_tensor.shape)

output_tensor = upsample.forward(input_tensor)
print("output_tensor.shape", output_tensor.shape)

dLdoutput_tensor = np.ones_like(output_tensor)
dLdinput_tensor = upsample.backward(dLdoutput_tensor)

print("input_tensor", input_tensor)
print("output_tensor", output_tensor)
print("dLdoutput_tensor", dLdoutput_tensor)
print("dLdinput_tensor", dLdinput_tensor)

print("input_tensor.shape", input_tensor.shape)
print("output_tensor.shape", output_tensor.shape)
print("dLdoutput_tensor.shape", dLdoutput_tensor.shape)
print("dLdinput_tensor.shape", dLdinput_tensor.shape)

def test_downsample1D(N, C, W, S):
    downsample = Downsample1D(S)
    # Only keep columns with index  $S*i$ , where  $i=0, \dots, W/S$ .

    input_tensor = np.random.uniform(0, 5, (N, C, W))
    print("input_tensor.shape", input_tensor.shape)

    output_tensor = downsample.forward(input_tensor)
    print("output_tensor.shape", output_tensor.shape)

    dLdoutput_tensor = np.ones_like(output_tensor)
    dLdinput_tensor = downsample.backward(dLdoutput_tensor)

    print("input_tensor", input_tensor)
    print("output_tensor", output_tensor)
    print("dLdoutput_tensor", dLdoutput_tensor)
    print("dLdinput_tensor", dLdinput_tensor)

    print("input_tensor.shape", input_tensor.shape)
    print("output_tensor.shape", output_tensor.shape)
    print("dLdoutput_tensor.shape", dLdoutput_tensor.shape)
    print("dLdinput_tensor.shape", dLdinput_tensor.shape)

if __name__ == '__main__':
    N = 1
    C = 3
    W = 5
    S = 3
    test_upsample1D(N, C, W, S)
    N = 1
    C = 3
    W = 13
    S = 3
    test_downsample1D(N, C, W, S)

```

